

Internet Packet Filter Management and Rectangle Geometry

David Eppstein*

Univ. of California, Irvine

S. Muthukrishnan†

AT&T Labs Research

Abstract

We consider rule sets for **internet packet routing and filtering**, where each rule consists of a range of source addresses, a range of destination addresses, a priority, and an action. A given packet should be handled by the action from the maximum priority rule that matches its source and destination. We describe **new data structures** for quickly finding the rule matching an incoming packet, in **near-linear space**, and a new algorithm for determining whether a rule set contains any conflicts, **in time** $\mathcal{O}(n^{3/2})$.

1 Introduction

The working of the current Internet and its posited evolution depend on efficient packet filtering mechanisms: databases of rules, maintained at various parts of the network, which use patterns to filter out sets of IP packets and specify actions to be performed on those sets. Typical filter patterns are based on packet header information such as the source or destination IP addresses. The actions to be performed depend on where the packet filtering is performed in the network. For example, at backbone routers, packet filters specify which interface or link to use when forwarding packets. In firewalls, packet filters specify whether to allow a connection. More generally, packet filters specify Quality-of-Service actions such as restricting certain classes of traffic to no more than a threshold bandwidth. This packet filtering mechanism — maintaining a database of filters with associated actions and applying them to IP packets as appropriate — underlies most crucial aspects of the Internet: correct routing, providing security, guaranteeing service level agreements between different subnets, billing based on traffic patterns, etc.

Implementing the packet filtering mechanism in the Internet involves sophisticated packet filter management tasks. In particular, we need *packet classification*, that is, given an IP packet with a specific header values, we need to determine which filter applies to that packet. We also need *filter conflict detection*, that is, we need to determine whether two or

more filters that apply to a packet specify conflicting actions. Conflicts are resolved by adding additional filters, so the filter database remains consistent. These are the fundamental packet filter management tasks governing the IP network performance.

In this paper, we present efficient algorithms for solving both of these packet filter management problems. Our approach is to solve the underlying abstract problem which, in each case, is naturally formulated as a geometric data structural problem. We focus on simple techniques suitable for highly efficient implementations, especially in our packet classification algorithms, because in the future we hope to explore implementations of them in practical applications. However our work provides theoretical asymptotic improvements as well.

The same abstract geometric data structural problems derived from these packet filtering applications arise independently in other important applications areas as well, and our results improve the best known results for those applications. In what follows, we describe the packet filter management problems (Section 1.1) and our results (Section 1.2), and provide an overview of our techniques (Section 1.3) before providing all the details (Sections 2 to 3). We will briefly describe the other application areas where our results are relevant in Section 1.3.

1.1 Packet Filter Management Problems

A packet filter i in IP networks is a collection of d -dimensional *ranges* $[l_i^1, r_i^1] \times \dots \times [l_i^d, r_i^d]$, an *action* A_i and a *priority* p_i . The precise nature of action is not relevant here except that we can determine if two actions A_i and A_j are in *conflict* (for example, if A_i is to allow the packet through the firewall and A_j is to disallow it, there is a conflict of action). Any IP packet P can be viewed as a d -dimensional vector of values $[P_1, \dots, P_d]$ summarizing the header information of the packet. A filter i *applies* to packet P if $P_j \in [l_i^j, r_i^j]$ for each $j \in [1, d]$.

Packet Classification Problem. A database F of filters is available for preprocessing. Each online query is a packet P , and the goal is to *classify* it, that is, to determine the filter of highest priority that applies to P . A related problem is to list all filters that apply to P .

*Dept. Inf. & Comp. Sci., Univ. of California, Irvine, CA 92697-3425. Email: eppstein@ics.uci.edu. Work performed in part while visiting AT&T.

†AT&T Labs, Shannon Laboratory, 180 Park Ave., Florham Park, NJ 07932. Email: muthu@research.att.com.

Filter Conflict Detection Problem. Given a database F of filters, determine if there exists any packet P such that of the filters of the highest priority that apply to P , any two of them specify actions that conflict. Related problems are to list all regions wherein conflicting P 's lie, or to list all conflicting pairs of filters.

Some remarks follow. Existing IP routers use destination based routing, that is, use filters with $d = 1$ specifying ranges of destination IP addresses. As the Internet evolves from being the best effort network as it is now to provide differentiated services, two or more IP header fields may be specified by a filter. Some proposals are underway to specify many fields such as source IP address, destination IP address, source port, destination port etc., while others are underway which seem to preclude using more than just the source and destination IP addresses, that is, $d = 2$ (in IPsec for example, the source or destination port numbers may not be revealed.) In the rest of this paper, we will assume $d = 2$ and the fields that are specified are source and destination IP addresses since that seems likely to be most prevalent and of immediate interest.

Filters typically specify IP address ranges as an IP address $a_1 \cdots a_{32}$ and a mask of certain number l of bits, that is, the range is $a_1 \cdots a_l 00 \cdots 0$ to $a_1 \cdots a_l 11 \cdots 1$. So these are not arbitrary ranges. Instead they are *hierarchical*, that is, if two ranges intersect, one is completely contained in the other. All our results will in fact work for arbitrary ranges in each dimension although some of our algorithms can be made simpler for implementation purposes if the ranges are hierarchical.

In both problems we will let n denote the number of filters in F . The value of n varies depending on where filtering is done: backbone routers may have hundreds of thousands of filters, firewalls may only have a few hundreds, etc. All numbers are integers in the range $[0, U - 1]$ — for IP addresses, this is currently $[0, 2^{32} - 1]$, but may go up to 2^{64} or higher in IPv6.

1.2 Our Results

Our main results are as follows.

- **Packet Classification Problem.** We present an algorithm for this problem with different tradeoffs for data structure space vs filtering time. In particular, we obtain very fast classification times with near-linear space: with $\mathcal{O}(n^{1+o(1)})$ space, classification takes $\mathcal{O}(\log \log n)$ time, or with $\mathcal{O}(n^{1+\epsilon})$ space, classification takes $\mathcal{O}(1)$ time.¹

¹For clarity, we have stated the results for $U = n^{\mathcal{O}(1)}$; bounds for general U appear later in the paper. Throughout this section, we assume $U = n^{\mathcal{O}(1)}$ for making comparisons with existing results.

- **Filter Conflict Detection Problem.** We present an $\mathcal{O}(n)$ space, $\mathcal{O}(n^{3/2})$ time algorithm for this problem. Straightforward $\mathcal{O}(n^2 \log n)$ time algorithms were the only known previous result.

The packet classification problem has been extensively studied with over a dozen papers in the premier networking conferences (INFOCOM and SIGCOMM) in the past few years (e.g., see references in [7]). Classification time is of paramount importance (for example, for backbone routers, filtering IP packets has to be done at the speed at which it forwards the packets, a blistering speed!). However, at such high speeds, *memory is very expensive* and the consensus in the networking community is that classification must be very fast, but that data structural space must be limited to the extent possible. The applied works in INFOCOM and SIGCOMM use near-linear space, but take time $\Omega(\log n)$ to classify each packet which they attempt to further speed up using large memory cache line etc. However, the golden standard has been the bound of $\Theta(\log \log n)$ that can be achieved for the $d = 1$. With the exception of [7], known algorithms for $d = 2$ fail to meet this bound. Our algorithmic result here meets this bound, but uses only $\mathcal{O}(n^{1+o(1)})$ space improving upon the $\Theta(n^{1+\epsilon})$ space needed by [7, 8] which is the previously best known result. Furthermore, our result is easily implementable; hence, it additionally holds promise as a practical packet classification solution.

The filter conflict detection problem has received attention only recently [1]. That work was primarily motivated by detecting security holes in firewalls. Filter databases in firewalls get modified by systems administrators manually or automatically (for example, when a host from inside a firewall requests a TCP connection with a host outside, a filter may be added to the firewall to enable the target host to open a TCP connection through the firewall). Conflicts arise quite naturally, and the task of the administrator is to resolve them appropriately. The work in [1] was motivated by this scenario. However, conflict detection helps in auditing filter databases [6] in general for ambiguities in routing, unfulfilled service guarantees etc., that is, in general where packet filter mechanism is employed. It is straightforward to solve this problem in $\mathcal{O}(n^2 \log n)$ time. Our main contribution here is in breaking the quadratic barrier and designing an $\mathcal{O}(n^{3/2})$ time algorithm.

1.3 Our Techniques and Other Applications of Our Results

Both the packet classification and the filter conflict detection problem can be thought of as *geometric problems* in which each rule is a 2-dimensional axis-parallel rectangle.² The packet classification problem can be viewed as *locating a*

²In d dimensions, they will be d -dimensional hyperrectangles.

point (the query IP packet header values) in a partition of space formed by overlaying these rectangles. The filter conflict detection problem is that of detecting certain overlapping regions among rectangles of highest priority that overlap a region. Our approach is to solve the underlying geometric data structural problems in the bounds quoted above. This has other immediate applications, for example to the problems in [8], giving the following new results: (1) faster multi-method lookup in object oriented languages and the first known efficient algorithm for auditing multi-method libraries, (2) improved matching algorithms for rectangular matching, wherein, for the first time, matching time is independent of the dictionary size while the space used is sub-quadratic in dictionary size, and (3) the first known optimal algorithm for approximately matching a pattern string with edit distance at most 1 in a text – the matching time is linear in the text size and preprocessed space is sublinear in the dictionary size. These three problems have extensive literature, and all these results are of independent interest. Readers are referred to [8] for details.

Our approach to solving the two packet classification relies on a standard plane-sweep approach to turn the static two-dimensional rectangle query problem into a dynamic one-dimensional problem, in which we maintain a dynamic set of intervals and must again query the maximum priority set element containing a query point. This one-dimensional problem must be solved *persistently*, so we can query previous versions of the data structure after the plane sweep has occurred. We solve this persistent one-dimensional problem using a data structure combining ideas from B-trees and segment trees.

Our approach to the filter conflict detection problem uses a technique related to an algorithm by Overmars and Yap [11] for *Klee's measure problem* (determining the volume of a union of rectangular blocks): we use a kD-tree [5] to divide the plane into rectangular cells, not containing any rectangle vertex, so that the rectangles intersecting any cell form *stripes* (i.e., rectangles that are unbounded in one dimension). The conflict detection problem can thus be reduced to determining a lower envelope of line segments, which can also be interpreted data structurally as an offline priority queue problem or graph theoretically as a minimum spanning tree verification problem. We solve this subproblem efficiently using a linear-time union-find data structure.

2 Fast Packet Classification Queries

As described above, packet classification can be viewed as an **orthogonal range querying problem**, in which we wish to find the maximum priority rectangle containing any query point. We now describe data structures for solving this problem efficiently.

2.1 Persistent Interval Queries

First, we consider a dynamic one-dimensional query problem: what is the maximum priority interval containing a query point among a dynamically changing set of intervals, having integer endpoints in the range $[0, U - 1]$. We assume without loss of generality that U is a power of two. Our data structure will be *partially persistent*: an update must be performed on the most recent version of the structure, but a query can refer to any prior version.

Our data structure will be parametrized by a value k , and will consist of *blocks* of $\mathcal{O}(2^k)$ memory words, each corresponding to information about an interval of values within the range $[0, U - 1]$. An update may create new blocks but will not change existing blocks. If a block corresponds to query values in the interval $[x, y]$, then by *subinterval* i we refer to the interval $[x + i(y - x)2^{-k}, x + (i + 1)(y - x)2^{-k} - 1]$. A persistent version of the data structure will be represented by a pointer to a block forming the top level of the data structure.

Each block contains the following information:

- A table $\text{opt}[i]$ of pointers to the **maximum-priority** interval in the dynamic set that contains subinterval i .
- A table $\text{pq}[i]$ of pointers to **priority queue** data structures for the intervals containing subinterval i .
- A table $\text{subblock}[i]$ of pointers to blocks representing the subset of dynamic intervals having endpoints in subinterval i . If a subinterval contains no endpoints, this pointer is null.

The priority queues are not used for queries, and so do not need to be maintained persistently. We will later see how to eliminate them altogether for problems derived from hierarchical rectangle sets.

LEMMA 2.1. *The data structure described above can find the maximum priority interval containing a query point in time $\mathcal{O}((\log U)/k)$.*

Proof. We answer a query simply by repeatedly following the pointer $\text{subblock}[i]$ for the subinterval i that contains the query point. For each block found via this chain of pointers, we look up the value $\text{opt}[i]$ and compare the priorities of the intervals found in this way.

Each successive block in the chain corresponds to an interval of size smaller by a 2^{-k} factor than the previous block, so the total number of blocks considered is $\log_{2^k} U = (\log U)/k$. For any interval $\mathcal{I}$ containing the query point there is a maximal block such that $\mathcal{I}$ contains the subinterval containing the query in that block; then by the assumption of maximality $\mathcal{I}$ must have an endpoint in the block and is a candidate for $\text{opt}[i]$. Therefore, the true maximum-priority interval containing the query is one of the ones found by the query, and the query algorithm is correct. $\square$

LEMMA 2.2. *The data structure described above can be updated in time $\mathcal{O}((2^k \log n)(\log U)/k)$ and space $\mathcal{O}(2^k(\log U)/k)$.*

Proof. To insert or delete an interval, we create a new copy of each block containing one of the endpoint intervals. By the same argument used to bound query time, there are at most $2(\log U)/k$ such blocks. For each copied block, we update the priority queues corresponding to subintervals containing the updated interval, copy pointers to these priority queues into the $pq[i]$ pointers of the new block, and use these priority queues to set each value of $opt[i]$. We then copy each pointer subblock[i] from the previous version of the block, except for the one or two subintervals containing the updated interval's endpoints, which are changed to point to the new blocks for those subintervals. Each update causes the creation of at most $2(\log U)/k$ new blocks, using space $\mathcal{O}(2^k(\log U)/k)$. Each update also changes $\mathcal{O}(2^k(\log U)/k)$ priority queues, in time $\mathcal{O}((2^k \log n)(\log U)/k)$. $\square$

We summarize the results of this section:

THEOREM 2.1. *For any k there exists a data structure for maintaining dynamic prioritized intervals in the range $[0, U - 1]$, and finding the maximum priority interval containing a query point in any persistent version of the data structure, in time $\mathcal{O}((\log U)/k)$ per query, time $\mathcal{O}((2^k \log n)(\log U)/k)$ per update, and space $\mathcal{O}(2^k(\log U)/k)$ per update.*

The $\log n$ factor in the update time can be reduced by building a segment tree of subintervals within each block, and maintaining a priority queue of the dynamic intervals corresponding to each canonical interval of the segment tree; we omit the details, since this factor does not form an important part of our overall running time and can (as detailed below) be avoided entirely for hierarchical rectangles.

2.2 Static to Dynamic Transformation

We use the dynamic data structure of the previous subsection to solve our static rectangle querying problem, as follows.

LEMMA 2.3. *Suppose we are given a set S of n integers in the range $[0, U - 1]$. Then for any x we can build a data structure which finds the largest predecessor in S of a given query integer, in space $\mathcal{O}(nx \log_x U)$ and query time $\mathcal{O}(\log_x U)$.*

Proof. Form a set of intervals $[i, U - 1]$ with priority i for $i \in S$. The maximum priority interval containing q has as its left endpoint the predecessor of q . Thus, we can use a static version of the data structure described in Theorem 2.1 (with $k = \log x$) to solve this problem. $\square$

Beame and Fich [4] provide matching $\Theta(\log \log n / \log \log \log n)$ upper and lower bounds for integer predecessor queries in polynomial space, and survey several previous results on the problem. Because of the reduction above, their lower bounds apply as well to the maximum priority interval and rectangle problems. Our results escape this lower bound by having a space bound that depends on U and not just on n .

THEOREM 2.2. *Given a set of n axis-aligned prioritized rectangles with coordinates in the range $[0, U - 1]$, and a parameter x , we can build a data structure of size $\mathcal{O}(nx \log_x U)$ which can find the maximum priority rectangle containing a query point in time $\mathcal{O}(\log_x U)$.*

Proof. We consider a left-right sweep of the rectangles by a vertical line; for each position of the sweep line we maintain a dynamic set of intervals formed by the intersections of the rectangles with the sweep line. This intersection changes only when the sweep line crosses the left or right boundary of a rectangle; at the left boundary we insert the y -projection of the rectangle and at the right boundary we delete it. With each rectangle boundary we store a pointer to the version of the data structure formed when crossing that boundary.

A query can be handled by using the integer predecessor data structure of Lemma 2.3 to find the x -coordinate of the nearest rectangle boundary to the right of the query point, and then performing a query in the corresponding version of the interval data structure. $\square$

In particular when $U = n^{\mathcal{O}(1)}$ we achieve query time $\mathcal{O}(\log \log n)$ in space $\mathcal{O}(n^{1+o(1)})$, or query time $\mathcal{O}(1)$ in space $\mathcal{O}(n^{1+\epsilon})$, while previous solutions used space $\Theta(n^{1+\epsilon})$ to achieve query time $\mathcal{O}(\log \log n)$ [8].

It is not difficult to modify our data structure to handle other decomposable queries, such as listing all rectangles containing the given query point, in similar time and space bounds.

For hierarchical rectangles, we can simplify the dynamic interval data structure by using insertion and undo operations instead of more general insertions and deletions, and by omitting the $pq[i]$ pointers and the priority queues they point to. An insertion can be handled by comparing the priority of the newly inserted interval to the values $opt[i]$ for the blocks containing the interval's endpoints. An undo can be handled simply by restoring the pointer to the top-level block to its previous version.

3 Conflict Detection

We say that a set of rules, represented by a set of rectangles with priorities, has a *conflict* if there exists a query point q such that there is not a unique maximum-priority rectangle containing q . Note that this is a stronger condition than the existence of an intersecting pair of equal-priority rectangles,

有多個最大排序的方形形 $\rightarrow$ 衝突

since a higher-priority rectangle could cover the intersection and avoid a conflict. As defined in the introduction, the filter conflict detection problem further **restricts conflicts to rules with conflicting actions**; the algorithms described here can be extended to cases where the actions can be partitioned into a small number of conflict types but **we omit the details**.

We would like to know whether a given set of prioritized rectangles has a conflict. A naive method would test **each pair of equal-priority rectangles to determine whether they conflict**, but this would not be efficient due to the difficulty of testing whether their intersection is covered by the union of higher priority rectangles. Less naively, the problem can be solved in near-quadratic time by querying each point determined by the horizontal boundary of one rectangle and the vertical boundary of another, or by constructing the arrangement of all the rectangles and using a priority queue to find the maximum priority rectangle(s) within each arrangement cell. We seek an even more efficient (subquadratic) solution.

3.1 Priority Queues, Lower Envelopes, and MST Verification

Consider the following three problems:

- Given is an offline sequence of $\mathcal{O}(n)$ integer priority queue operations: insert or delete a value in the set $\{0, 1, \dots, n-1\}$ and query the minimum value. How quickly can one answer all the queries?
- Given is a set of horizontal line segments (Figure 1, left), each endpoint of which has coordinates in the set $\{0, 1, \dots, n-1\}$. How quickly can one construct the lower envelope of the line segments? That is, if we think of each line segment as representing the graph of a (constant) function defined over a portion of the x -axis, what is the (piecewise constant) minimum of these functions (Figure 1, right)?
- Given is a graph, in which the minimum spanning tree is a given path, and in which all edges have weights in the set $\{0, 1, \dots, n-1\}$. How quickly can one determine, for each edge in the path, which edge would replace it in the MST if the path edge were deleted?

It is not difficult to see that in fact these problems are equivalent to each other: the insertion times, deletion times, and priorities in the offline priority queue correspond respectively to the x -coordinates of the left endpoints, x -coordinate of the right endpoints, and y -coordinates of the horizontal line segments, which correspond respectively to the first vertex (according to the path order), second vertex, and weight of the non-MST edges in the graph.

Aho, Hopcroft, and Ullman [2, pp. 139–141] describe an algorithm for a similar offline priority queue problem,

however their problem involves delete-minimum operations rather than deletions of particular values. Although the best replacement edge for each non-MST edge can be found in linear time [10], the fastest known algorithm for finding the best replacement for each MST edge (without the integer restriction) remains Tarjan’s slightly superlinear one [12].

LEMMA 3.1. *The three problems described above can be solved in linear time.*

Proof. We consider the minimum spanning tree verification formulation of the problem, and consider the non-tree edges in sorted order by weight. Our algorithm finds the replacements for each path edge in a certain order; when a path edge’s replacement is found we reduce the size of the graph by contracting that edge. This contraction clearly does not change the replacement for the remaining edges. We use a union-find data structure to keep track of the relation between the original graph vertices and the vertices of the contracted graph. Since the contractions will be performed along the edges of a fixed tree (namely, the given path), we can use the linear-time union-find data structure of Gabow and Tarjan [9] or its recent simplification by Alstrup et al. [3].

Our algorithm, then, simply performs the following steps for each edge (u, v) , in sorted order by edge weight: for each uncontracted edge (x, y) remaining in the path between u and v , set that edge’s replacement to (u, v) , contract the edge, and unite x and y in the union-find data structure.

The time per edge (u, v) is a constant, plus a term proportional to the number of path edges contracted as a result of processing edge (u, v) . Since each edge can only be contracted once, the total time is linear. $\square$

The technique readily extends to finding best replacement edges for graphs where the MST is not a path.

For our application to conflict detection, we also need to know whether there were any ambiguities in the above process; that is, whether any of the offline min operations can return more than one equal minimum value. This is essentially the same as our original conflict detection problem in one dimension rather than two. One way to solve this is to apply the above algorithm twice, once with an arbitrary tie-breaking order imposed on equal weight edges, and once again with the reverse order imposed, and test whether the two applications of the algorithm produce the same assignment of replacement edges.

3.2 Stripes

We first describe an efficient algorithm for conflict detection in the special case that each rectangle is a *stripe*; that is, either its vertical extent or its horizontal extent is the entire space $[0, U-1]$. We do not expect such a restricted case to occur in our application, but it forms an important subroutine for our more general algorithm.

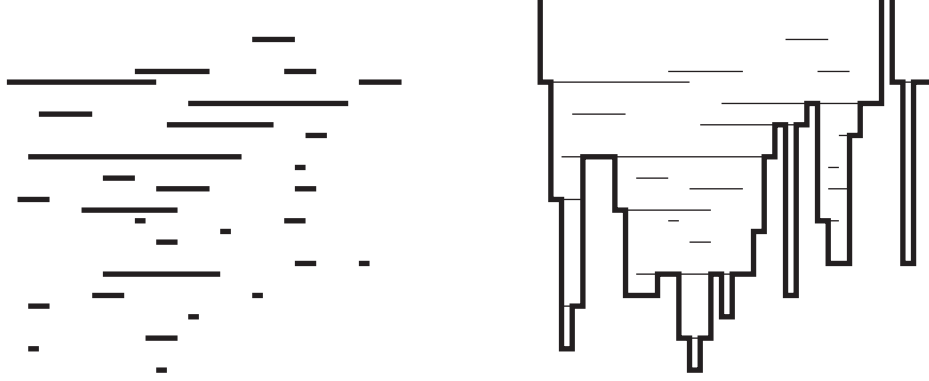


Figure 1: A set of horizontal line segments (left) and its lower envelope (right).

We classify stripes into three types:

- A **horizontal** stripe has x -extent $[0, U - 1]$ and y -extent a proper subset of $[0, U - 1]$.
- A **vertical** stripe has x -extent a proper subset of $[0, U - 1]$ and y -extent $[0, U - 1]$.
- A **universal** stripe has both x -extent and y -extent equal to the entire space $[0, U - 1]$.

LEMMA 3.2. *Let a collection of prioritized stripes be given, together with sorted orderings of all stripes according to their priorities, the horizontal boundaries of horizontal stripes according to their y -coordinates, and the vertical boundaries of vertical stripes according to their x -coordinates. Then we can detect a conflict in this set of stripes in linear time.*

Proof. We first partition the space $[0, U - 1]^2$ into horizontal stripes, according to the maximum-priority horizontal input stripe covering each point in the space; essentially this is just the lower envelope computation of Lemma 3.1. Let m_h denote the minimum priority occurring in this partition. Similarly, we partition the space into vertical stripes according to the maximum-priority vertical input stripe covering each point, and let m_v denote the minimum priority occurring in this partition. Finally, we let m_u denote the maximum priority of any universal stripe. (We set m_h , m_v , or m_u to $-\infty$ if the corresponding set of stripes is empty.)

We then use this information to search for conflicts, as follows, depending on the types of the two conflicting stripes:

- To find a conflict between two horizontal stripes, if one exists, test whether there exists an ambiguity in the construction of the horizontal partition, as discussed below Lemma 3.1. If there is such an ambiguity, let p_h

denote the maximum priority of any ambiguity. Then a conflict exists if and only if $p_h \geq \max\{m_v, m_u\}$. Similarly we can find a conflict between two vertical stripes by letting p_v denote the maximum priority of an ambiguity in the vertical partition, and testing whether $p_v \geq \max\{m_h, m_u\}$.

- A conflict between two universal stripes exists if and only if some two or more universal stripes have priority m_u , and if $m_u \geq \max\{m_h, m_v\}$.
- A conflict between a universal and a horizontal stripe exists if and only if m_u is also the priority of one of the stripes in the horizontal partition, and $m_u \geq m_v$. Similarly a conflict between a universal and a vertical stripe exists if and only if m_u is also the priority of one of the stripes in the vertical partition, and $m_u \geq m_h$.
- A conflict between a horizontal stripe and a vertical stripe exists if and only if there is a priority $p \geq m_u$ that appears both in the horizontal and the vertical partition.

Thus, the problem has been reduced to a constant number of comparisons, together with two more complex operations: determining whether m_u appears in either of two sets of priorities, and determining the intersection of those two sets. Since we know the sorted order of the priorities, we can represent them by values in the range $[0, n - 1]$ and use a simple bitmap to perform these membership and intersection tests in linear total time. $\square$

3.3 kD-tree

A *kD-tree* [5] of a set of points is a hierarchical partition into rectangular *cells*, formed as follows:

- The root of the hierarchy is a bounding box for the point set.
- If a cell at an even level of the hierarchy contains one or more points in its interior, then it is split into two

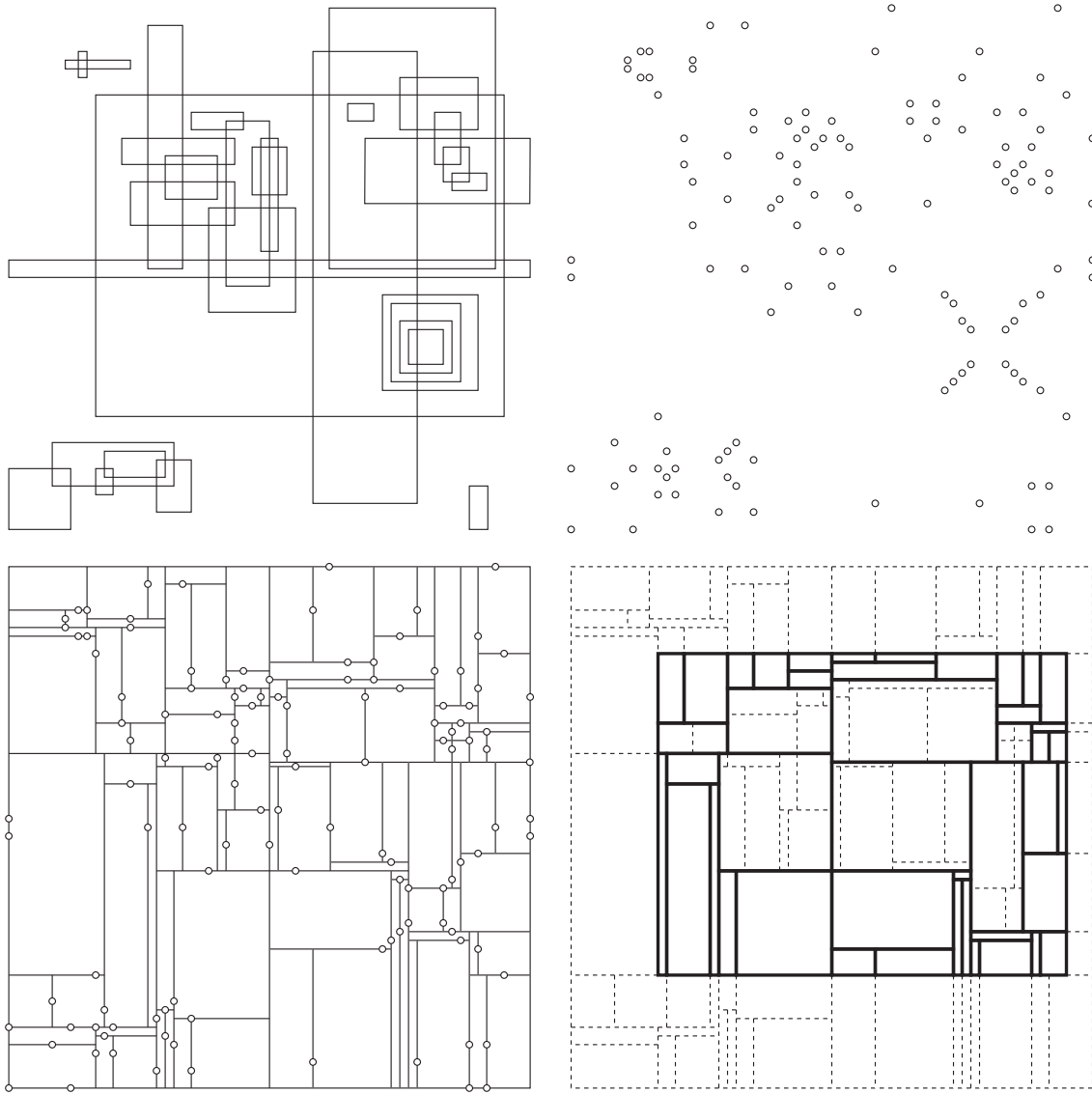


Figure 2: kD -tree for a set of rectangles. Upper left: the rectangles. Upper right: their vertices. Lower left: kD -tree for the vertices. Lower right: maximal kD -tree cells partition an input rectangle.

smaller cells by a vertical line through the point with the **median x-coordinate**.

- If a cell at an odd level of the hierarchy contains one or more points in its interior, then it is split into two smaller cells by a horizontal line through the point with the **median y-coordinate**.

If a cell contains an even number of points, either of the two median points can be used to determine its split line. The leaf cells of the kD -tree form a partition of the bounding box into $\mathcal{O}(n)$ empty rectangles. Since each split divides its point set in half, the number of levels of the hierarchy is at most $\log_2 n$ (it can be smaller if several points are contained in a single split line).

LEMMA 3.3. *Any vertical or horizontal line cuts $\mathcal{O}(\sqrt{n})$ cells at all levels of the kD -tree for a set of $\mathcal{O}(n)$ points.*

Proof. If the line is horizontal (vertical), the number of cells cut by the line at most doubles at every even (odd) level of the kD -tree construction, and remains unchanged at every odd (even) level. The result follows from the $\log_2 n$ bound on the number of levels in the tree. $\square$

In our conflict detection algorithm, we will use kD -trees defined on the set of corners of the input rectangles (Figure 2). We say that an input rectangle *covers* a kD -tree cell if the cell is completely contained in the rectangle. We define a *maximal covered cell* for a given rectangle to be a cell that is covered by the rectangle, but for which the cell's parent is not covered. We say that a rectangle *crosses* a cell if it has a nonempty intersection with the interior of the cell but does not cover it.

LEMMA 3.4. *Any rectangle has $\mathcal{O}(\sqrt{n})$ crossed cells and $\mathcal{O}(\sqrt{n})$ maximal covered cells at all levels in the kD -tree.*

Proof. The bound on the number of crossed cells follows immediately from Lemma 3.3. The parent of a maximal covered cell must be crossed, and each crossed cell can have at most one maximal covered child, so the number of maximal covered cells is also $\mathcal{O}(\sqrt{n})$. $\square$

3.4 The Conflict Detection Algorithm

Clearly, if a set of rectangles has a conflict, then this conflict must occur within at least one of the leaf cells of a kD -tree. Further, since the leaf cells contain no rectangle corners, each rectangle acts like a stripe within any such cell: it extends either the full width or the full height of the cell. Thus, we can perform conflict detection by building a kD -tree and applying our stripe conflict detection algorithm to each cell.

THEOREM 3.1. *Given a set of n prioritized rectangles, we can determine whether the set has a conflict in time $\mathcal{O}(n^{3/2})$ and space $\mathcal{O}(n)$.*

Proof. We build a kD -tree of the rectangle vertices (this can be done in time $\mathcal{O}(n \log n)$) and perform a depth first traversal of the tree. As we traverse the tree, we maintain at each cell of the traversal the following information:

- The maximum priority of a rectangle covering the cell, and one or (if they exist) two rectangles having that maximum priority.
- A list of the rectangles crossing the cell, sorted by priority.
- A sorted list of the horizontal boundaries of the rectangles that cross the cell.
- A sorted list of the vertical boundaries of the rectangles that cross the cell.

When the traversal reaches a cell C , we can determine which of rectangles cross or cover the children of C , and extract the sorted sublists for its two children, in time linear in the number of rectangles crossing C . We also find the set of rectangles that cross C but maximally cover one of its children, scan this set for the maximum priority, and use this information (together with the maximum priority of a rectangle covering C) to determine the maximum priority of a rectangle covering each child.

When the traversal reaches a leaf cell, we apply the algorithm of Lemma 3.2 to test whether the cell contains a conflict.

While one child of a cell C is being processed recursively, we store with C only the portions of the sorted lists that have not been passed to that child, so that each rectangle or rectangle edge is stored in one of the lists only at a single level of the tree, keeping the total space linear. All operations performed when traversing a cell take time linear in the number of rectangles crossing or maximally covering the cell, so by Lemma 3.4 the total time is $\mathcal{O}(n\sqrt{n})$. $\square$

4 Concluding Remarks

We have considered the two fundamental packet filter management problems in IP networks, namely, packet classification and filter conflict detection, for the two dimensional case of immediate interest. For the packet classification problem, we present a simple algorithm that takes $\mathcal{O}(\log \log n)$ time to classify packets matching the best known bounds for the one dimensional case, and improving upon the space needed by currently known solutions. For the filter conflict detection problem, our solution is the first sub-quadratic time algorithm.

Our packet classification algorithm may well turn out to be better than existing ones in practice, too. We fully intend to test that possibility. However, the task is not one of merely implementing our algorithm and comparing against the known ones. Since the study of packet classification is

quite mature in the networking communities, we need to do a careful job adapting our solution (where to make best use of large memory cache line, how to combine hardware and software solutions, how to exploit the properties of rule sets to isolate small, hard subproblems where our solution will be useful, etc). Engineering such tradeoffs is best explored in a separate paper.

Dynamic versions of the packet filter management problem are open, as are extensions of our results to higher dimensional query problems.

References

- [1] H. Adiseshu, S. Suri, and G. Parulkar. Detecting and resolving packet filter conflicts. In *Proc. INFOCOM*, volume 3, pages 1203–1212. IEEE, March 2000.
- [2] A. V. Aho, J. E. Hopcroft, and J. D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
- [3] S. Alstrup, J. P. Secher, and M. Spork. Optimal on-line decremental connectivity in trees. *Inf. Proc. Lett.*, 64(4):161–164, November 1997.
- [4] P. Beame and F. E. Fich. Optimal bounds for the predecessor problem. In *Proc. 31st ACM Symp. Theory of Computing*, pages 295–304, May 1999.
- [5] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Commun. ACM*, 18(9):509–517, September 1975.
- [6] D. Decasper, Z. Dittia, G. Parulkar, and B. Plattner. Router plugins: a software architecture for next-generation routers. *IEEE/ACM Trans. Networking*, 8(1):2–15, February 2000.
- [7] A. Feldmann and S. Muthukrishnan. Tradeoffs for packet classification. In *Proc. INFOCOM*, volume 3, pages 1193–1202. IEEE, March 2000.
- [8] P. Ferragina, S. Muthukrishnan, and M. de Berg. Multi-method dispatching: a geometric approach with applications to string matching problems. In *Proc. 31st ACM Symp. Theory of Computing*, pages 383–491, May 1999.
- [9] H. N. Gabow and R. E. Tarjan. A linear-time algorithm for a special case of disjoint set union. *J. Comput. Sys. Sci.*, 30(2):209–221, April 1985.
- [10] V. King. A simpler minimum spanning tree verification algorithm. *Algorithmica*, 18(2):263–270, June 1997.
- [11] M. H. Overmars and C. K. Yap. New upper bounds in Klee’s measure problem. *SIAM J. Comput.*, 20:1034–1045, 1991.
- [12] R. E. Tarjan. Applications of path compressions on balanced trees. *J. ACM*, 26(4):690–715, October 1979.